Lab04

REPORT

Applied Machine Learning - COS30082

Student Name: Truong Ngoc Gia Hieu
Student ID (VietNam): SWS01217
StudentID (Australia): 105565520

Swinburne University of Technology

A. Data Loading and Exploratory Data Analysis 3
1.1 Data Loading 3
1.2 Exploratory Data Analysis 3
B. Data Preparation 6
2.1 Image Resizing and Normalization 6
2.2 Train-Validation Split 7
2.3 Dataset Summary and Samples 7
C. Convolutional Neural Network (CNN) Model 8
3.1 Base CNN Architecture 8
3.2 Training and Evaluation 13
3.3 Testing 14
D. Conclusion 15

A. Data Loading and Exploratory Data Analysis

1.1 Data Loading

First and foremost, I had to download **the** dataset consisting of the ***seg_train*** and ***seg_test*** folders from the following Kaggle link: *https://www.kaggle.com/datasets/puneet6060/intel-image-classification*. Then, I uploaded them to ***Google Drive*** to easily import the data into ***Google Colab*** due to the large file size.

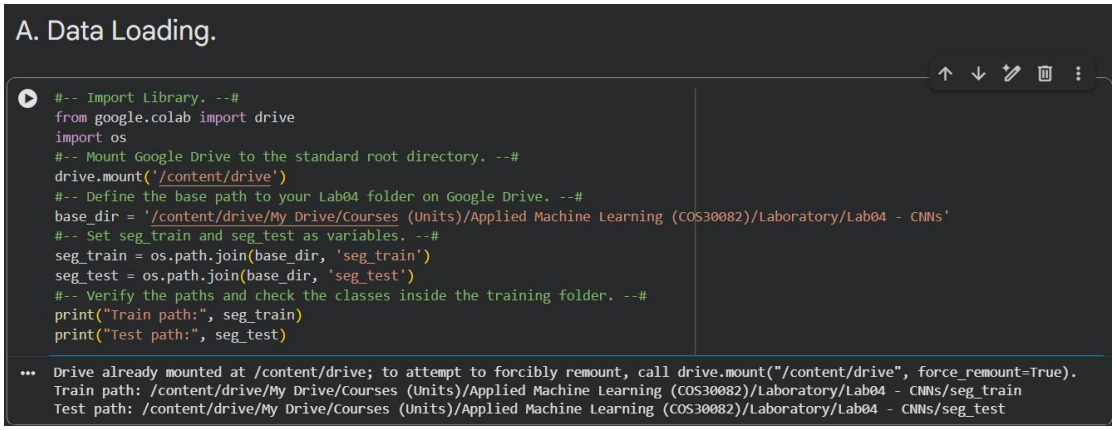


Figure 1: Google Drive Mounting & Path Configuration.

1.2 Exploratory Data Analysis

Next, in the Exploratory Data Analysis section, the number of images in the training and testing datasets was examined to analyze the class distribution and verify the total sample count for each category.

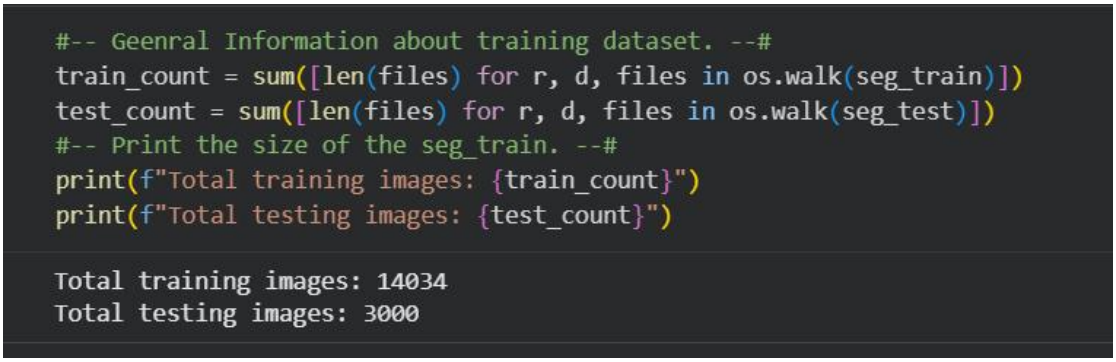


Figure 2: Dataset Training and Testing Size Overview.

Then, the following characteristics of each image were examined: resolution, color channels, file format, and file size.

2.2 Image Metadata.

```python
import os
from PIL import Image
#-- Get a sample image from the training dataset. --#
sample_path = os.path.join(seg_train, 'buildings', os.listdir(os.path.join(seg_train, 'buildings'))[0])
img = Image.open(sample_path)
print(f"- Image Resolution: {img.size[0]} x {img.size[1]} pixels")
print(f"- Color Channels: {len(img.getbands())}")
print(f"- File Format: {img.format}")
print(f"- File Size: {os.path.getsize(sample_path) / 1024:.2f} KB")
```

```
- Image Resolution: 150 x 150 pixels
- Color Channels: 3
- File Format: JPEG
- File Size: 14.30 KB
```

Figure 3: Image Metadata and Properties Overview.

```python
import os
import matplotlib.pyplot as plt
from PIL import Image
#-- Analyze the number of images per class (Class Distribution). --#
print("--- CLASS DISTRIBUTION ---")
class_names = sorted(os.listdir(seg_train))
class_counts = {}
for name in class_names:
    path = os.path.join(seg_train, name)
    if os.path.isdir(path):
        count = len(os.listdir(path))
        class_counts[name] = count
        print(f"- {name}: {count} images")

#-- Visualize sample images from each class (Sample Image Grid). --#
print("-" * 27)
plt.figure(figsize=(12, 8))
for i, name in enumerate(class_names):
    path = os.path.join(seg_train, name)
    if os.path.isdir(path):
        img_name = os.listdir(path)[0]
        img_path = os.path.join(path, img_name)

        img = Image.open(img_path)
        plt.subplot(2, 3, i + 1)
        plt.imshow(img)
        plt.title(f"Class: {name}\nResolution: {img.size}")
        plt.axis('off')

plt.tight_layout()
plt.show()
```

```
--- CLASS DISTRIBUTION ---
- buildings: 2191 images
- forest: 2271 images
- glacier: 2404 images
- mountain: 2512 images
- sea: 2274 images
- street: 2382 images
---------------------------
```

Figure 4: Training Class Distribution Analysis.

2.4 Quality and Variances.

[16] ✓ 0s

```
import os
import numpy as np
from PIL import Image
pixel_sums, pixel_squared_sums, total_pixels = np.zeros(3), np.zeros(3), 0
#-- Take 20 images ine ach folder to calculate. --#
for class_name in sorted(os.listdir(seg_train)):
    class_path = os.path.join(seg_train, class_name)
    if os.path.isdir(class_path):
        files = os.listdir(class_path)[:20]
        for img_name in files:
            if img_name.lower().endswith(('png', 'jpg', 'jpeg')):
                with Image.open(os.path.join(class_path, img_name)) as img:
                    img_np = np.array(img.convert('RGB')) / 255.0
                    total_pixels += img_np.shape[0] * img_np.shape[1]
                    pixel_sums += img_np.sum(axis=(0, 1))
                    pixel_squared_sums += (img_np ** 2).sum(axis=(0, 1))

mean = pixel_sums / total_pixels
std = np.sqrt((pixel_squared_sums / total_pixels) - (mean ** 2))

print(f"Mean (RGB): {mean}")
print(f"Std (RGB): {std}")
```

```
Mean (RGB): [0.43073082 0.45557373 0.44804178]
Std (RGB): [0.28034189 0.27317377 0.29544243]
```

Figure 5: Pixel-Level Statistics and Normalization Parameters..

2.5 Visual Inspection.

```
import os
import matplotlib.pyplot as plt
from PIL import Image
#-- Visualize a sample image from each class. --#
class_names = sorted(os.listdir(seg_train))
plt.figure(figsize=(10, 6))

for i, class_name in enumerate(class_names):
    class_path = os.path.join(seg_train, class_name)
    if os.path.isdir(class_path):
        img_name = os.listdir(class_path)[0]
        img_path = os.path.join(class_path, img_name)

        img = Image.open(img_path)
        plt.subplot(2, 3, i + 1)
        plt.imshow(img)
        plt.title(class_name)
        plt.axis('off')

plt.tight_layout()
plt.show()
```

Figure 6: Sample Image Grid for Visual Inspection.

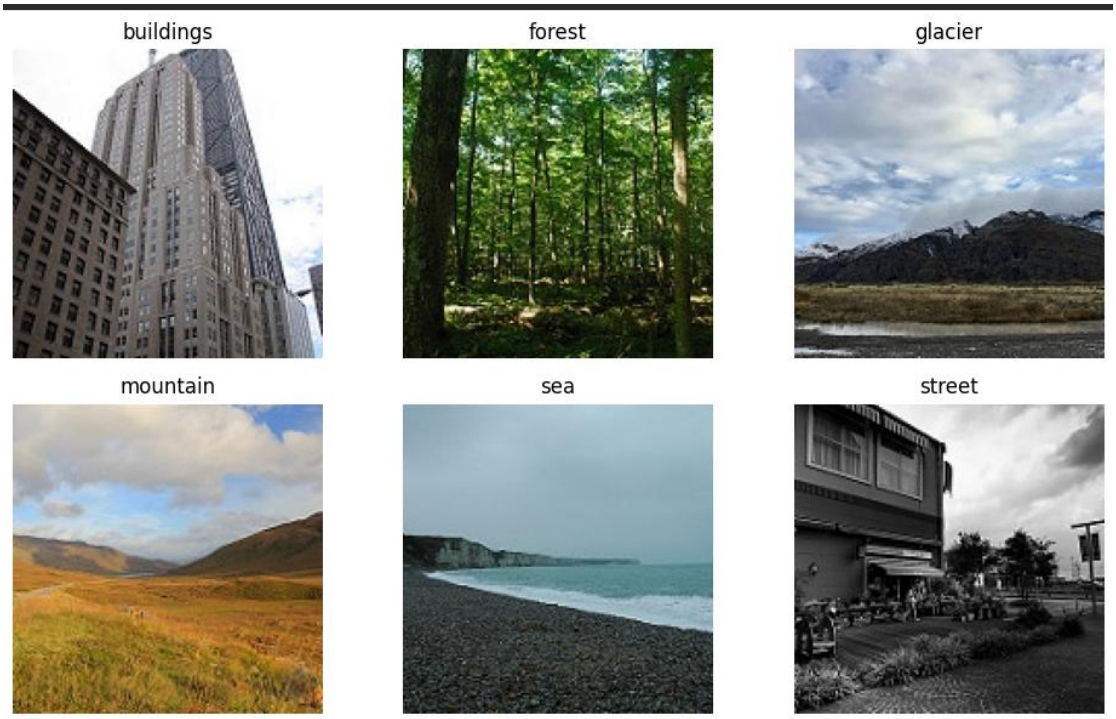


Figure 7: Sample Images Visualisation for Each Class.

B. Data Resizing and Normalization

In this section, the task requires students to load the dataset, resize images to 150x150 pixels, normalize pixel values to the range [0, 1], and split the training set into training and validation subsets.

2.1 Image Resizing and Normalization

```
#---- IMAGE RESIZING. ----#
#-- Import Python Library. --#
import tensorflow as tf
#-- Define dataset configuration parameters. --#
BATCH_SIZE = 32
IMG_SIZE = (150, 150)
#-- Load seg_train folder and automatically resize the training dataset to 150x150 pixels. --#
train_ds = tf.keras.utils.image_dataset_from_directory(seg_train, image_size=IMG_SIZE, batch_size=BATCH_SIZE, label_mode='categorical')
#-- Load seg_test folder and automatically resize the testing dataset to 150x150 pixels. --#
test_ds = tf.keras.utils.image_dataset_from_directory(seg_test, image_size=IMG_SIZE, batch_size=BATCH_SIZE, label_mode='categorical')
print(f"Image resizing completed successfully. Target resolution: {IMG_SIZE}")
```

```
Found 14034 files belonging to 6 classes.
Found 3000 files belonging to 6 classes.
Image resizing completed successfully. Target resolution: (150, 150)
```

Figure 8: Dataset Loading and Resizing Output.

After resizing the image to 150x150, pixel normalization is applied to scale raw pixel values from the [0, 255] range down to [0, 1]. Notably, this crucial preprocessing step stabilizes gradient updates and significantly accelerates model convergence during training.

After resizing the image to 150x150, pixel normalization is applied to scale values from [0, 255] down to [0, 1]. Notably, this crucial preprocessing step stabilizes gradient updates and accelerates model convergence during training.

```
#---- IMAGE NORMALIZATION. ----#
#-- Define normalization function using tf.keras.layers.Rescaling --#
normalization_layer = tf.keras.layers.Rescaling(1./255) #-- Change from [0,255] to [0,1]. --#
train_ds = train_ds.map(lambda x, y: (normalization_layer(x), y))
test_ds = test_ds.map(lambda x, y: (normalization_layer(x), y))

print("Pixel normalization completed successfully.")
```

```
Pixel normalization completed successfully.
```

Figure 9: Image Pixel Normalization Code and Execution Output.

2.2 Train-Validation Split

Subsequently, the training dataset is partitioned into training and validation subsets. A 20% validation split (val_ds) is created to monitor model performance after each epoch, ensuring early detection and prevention of overfitting.

3.2 Train-Validation Split.

```
#-- Count total number of batch in the training folder and multiply with 0.2 (20%) to return integer number. --#
val_size = int(train_ds.cardinality().numpy() * 0.2)
val_ds = train_ds.take(val_size) #-- Take the rest batch for training. --#
train_ds = train_ds.skip(val_size)
print(f"Training batches: {train_ds.cardinality().numpy()}")
print(f"Validation batches: {val_ds.cardinality().numpy()}")
```

```
Training batches: 226
Validation batches: 56
```

Figure 10: Train-Validation Split Code and Batch Output.

2.3 Dataset Summary and Sample

In this part, the given dataset comprises natural and urban scenes categorized into six distinct classes: buildings, forest, glacier, mountain, sea, and street. In terms of visual inspection of the data structure, a grid of sample images alongside their corresponding labels is plotted below, confirming successful data loading from the directory.

```
import pandas as pd

#---- 1. GENERAL INFORMATION SUMMARY TABLE ----#
summary_data = {
    "Dataset Attribute": ["Name", "Total Classes", "Class Categories", "Target Resolution", "Split Ratio"],
    "Details": [
        "Intel Image Classification",
        "6 Classes",
        ", ".join(class_names),
        "150x150 pixels",
        "80% Train / 20% Validation"
    ]
}
summary_df = pd.DataFrame(summary_data)
print("=== DATASET SUMMARY TABLE ===")
display(summary_df)
```

```
=== DATASET SUMMARY TABLE ===
```

	Dataset Attribute	Details
0	Name	Intel Image Classification
1	Total Classes	6 Classes
2	Class Categories	buildings, forest, glacier, mountain, sea, street
3	Target Resolution	150x150 pixels
4	Split Ratio	80% Train / 20% Validation

Figure 11: Summary Table of Dataset Attributes and Configuration.

```python
import matplotlib.pyplot as plt
import numpy as np

#---- 2. QUICK SAMPLE IMAGES VISUALIZATION ----#
for images, labels in train_ds.take(1):
    img_batch = images.numpy()
    label_batch = labels.numpy()
    break

plt.figure(figsize=(6, 6))
for i in range(4):
    ax = plt.subplot(2, 2, i + 1)
    plt.imshow(img_batch[i])
    class_idx = np.argmax(label_batch[i])
    plt.title(class_names[class_idx])
    plt.axis("off")

plt.tight_layout()
plt.show()
print("Dataset sample visualization completed.")
```



Figure 12: Sample Dataset Images.

C. Convolutional Neural Network (CNN) Model

3.1 Base CNN Architecture

In the thirs section of the report, a baseline Convolutional Neural Network (CNN) architecture is implemented to establish a benchmark for image classification. Furthermore, the model extracts spatial features through sequential convolutional and pooling layers before classification. The model implementation and structural layout are presented below.

4.2 (Layer 1): Input layer.

```
#-- Import Liarary. --#
import tensorflow as tf
from tensorflow.keras import layers
#---- Input Layer ----#
input_layer = layers.Input(shape=(150, 150, 3)) #-- Each image has 150x150x3 = 67500 pixels with three channels. --#
print("Input layer defined successfully with shape:", input_layer.shape)
```

```
Input layer defined successfully with shape: (None, 150, 150, 3)
```

Figure 13: Input Layer.

4.2 (Layer 2): First Convolutional Layer.

```
#-- First Convolutional Layer. --#
conv_1 = layers.Conv2D(32, (3, 3), padding='same')
x1 = conv_1(input_layer)
```

```
x1
```

```
<KerasTensor shape=(None, 150, 150, 32), dtype=float32, sparse=False, ragged=False, name=keras_tensor_3>
```

Figure 14: First Convolutional Layer.

4.3 (Layer 3): First Activation Layer.

```
relu_1 = layers.Activation('relu')
x2 = relu_1(x1)
```

```
x2
```

```
<KerasTensor shape=(None, 150, 150, 32), dtype=float32, sparse=False, ragged=False, name=keras_tensor_4>
```

Figure 15: First Activation Layer.

4.4 (Layer 4): First Pooling Layer.

```
# Cell 4: First Pooling Layer
pool_1 = layers.MaxPooling2D((2, 2))
x3 = pool_1(x2)
```

```
x3
```

```
<KerasTensor shape=(None, 75, 75, 32), dtype=float32, sparse=False, ragged=False, name=keras_tensor_5>
```

Figure 16: First Pooling Layer.

LEVEL 2: Learn more complex structures and shapes.

4.5 (Layer 5): Second Convolutional Layer.

```
# Cell 5: Second Convolutional Layer
conv_2 = layers.Conv2D(64, (3, 3), padding='same')
x4 = conv_2(x3)
```

```
x4
```

```
<KerasTensor shape=(None, 75, 75, 64), dtype=float32, sparse=False, ragged=False, name=keras_tensor_6>
```

Figure 17: Second Convolutional Layer.

4.6 (Layer 6): Second Activation Layer (ReLU).

```
relu_2 = layers.Activation('relu')
x5 = relu_2(x4)
```

```
x5
```

```
<KerasTensor shape=(None, 75, 75, 64), dtype=float32, sparse=False, ragged=False, name=keras_tensor_7>
```

Figure 18: Second Activation Layer.

4.7 (Layer 7): Batch Normalization Layer.

```
bn_2 = layers.BatchNormalization()
x6 = bn_2(x5)
```

```
x6
```

```
<KerasTensor shape=(None, 75, 75, 64), dtype=float32, sparse=False, ragged=False, name=keras_tensor_8>
```

Why Batch Normalization starts from Layer 2: Batch Normalization is introduced from the second convolutional block onward to stabilize internal covariate shift as feature representations grow deeper and more complex, while the initial layer focuses on extracting raw low-level spatial features.

Figure 18: Batch Normolization Layer.

4.8 (Layer 8): Second Pooling.

```
pool_2 = layers.MaxPooling2D((2, 2))
x7 = pool_2(x6)
```

```
x7
```

```
<KerasTensor shape=(None, 37, 37, 64), dtype=float32, sparse=False, ragged=False, name=keras_tensor_10>
```

Figure 19: Second Pooling Layer.

4.9 (Layer 9): Third Convolutional layer.

```
conv_3 = layers.Conv2D(128, (3, 3), padding='same')
x8 = conv_3(x7)
```

```
x8
```

```
<KerasTensor shape=(None, 37, 37, 128), dtype=float32, sparse=False, ragged=False, name=keras_tensor_11>
```

Figure 20: Third Convolutional Layer.

4.10 (Layer 10): Third Activation Layer.

```
relu_3 = layers.Activation('relu')
x9 = relu_3(x8)
```

```
x9
```

```
<KerasTensor shape=(None, 37, 37, 128), dtype=float32, sparse=False, ragged=False, name=keras_tensor_12>
```

Figure 21: Third Activation Layer.

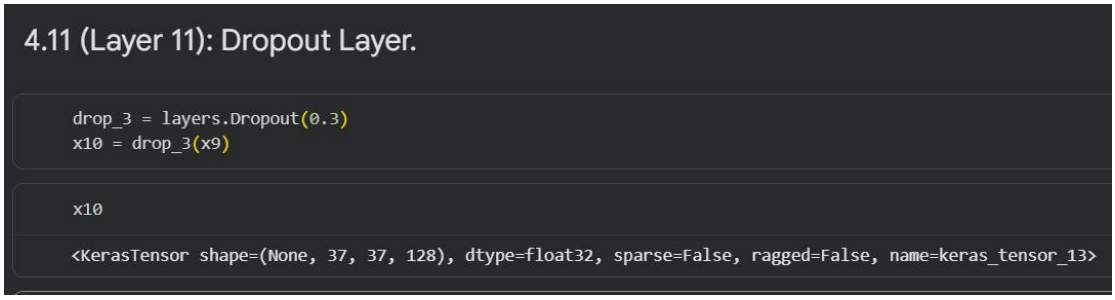


Figure 22: Dropout Layer.

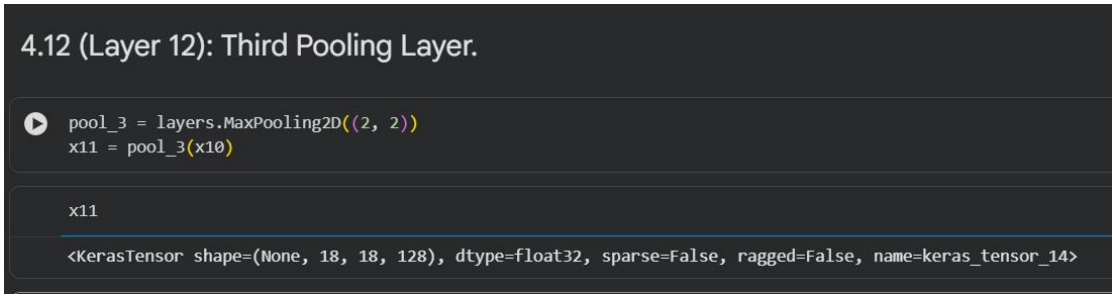


Figure 23: Third Pooling Layer.

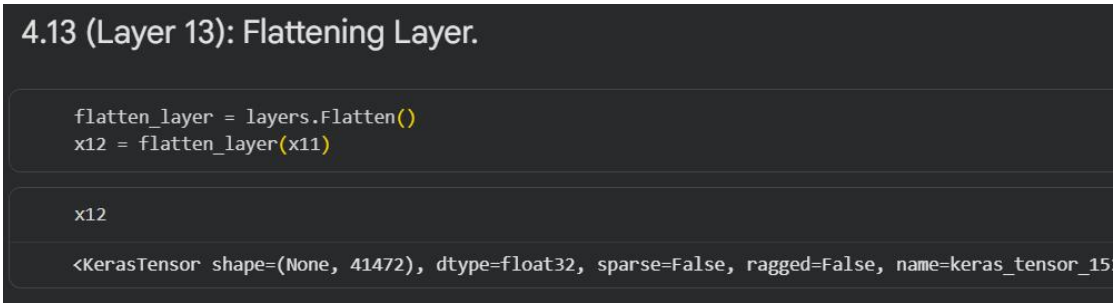


Figure 24: Flattening Layer.

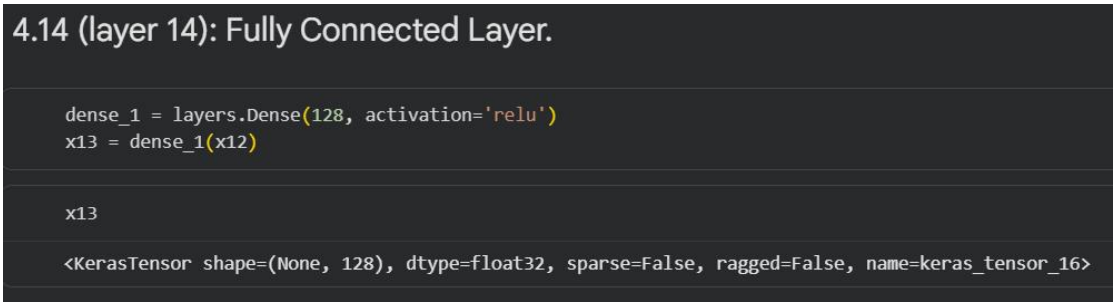


Figure 25: Fully Connected Layer.



Figure 26: Output Layer.

```
from tensorflow.keras import models

model = models.Model(inputs=input_layer, outputs=outputs, name="Base_CNN_Model")
model.summary()
```

Model: "Base_CNN_Model"

Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 150, 150, 3)	0
conv2d (Conv2D)	(None, 150, 150, 32)	896
activation (Activation)	(None, 150, 150, 32)	0
max_pooling2d (MaxPooling2D)	(None, 75, 75, 32)	0
conv2d_1 (Conv2D)	(None, 75, 75, 64)	18,496
activation_1 (Activation)	(None, 75, 75, 64)	0
batch_normalization (BatchNormalization)	(None, 75, 75, 64)	256
max_pooling2d_2 (MaxPooling2D)	(None, 37, 37, 64)	0
conv2d_2 (Conv2D)	(None, 37, 37, 128)	73,856
activation_2 (Activation)	(None, 37, 37, 128)	0
dropout (Dropout)	(None, 37, 37, 128)	0
max_pooling2d_3 (MaxPooling2D)	(None, 18, 18, 128)	0
flatten (Flatten)	(None, 41472)	0
dense (Dense)	(None, 128)	5,308,544
dense_1 (Dense)	(None, 6)	774

Total params: 5,402,822 (20.61 MB)
Trainable params: 5,402,694 (20.61 MB)
Non-trainable params: 128 (512.00 B)

Figure 27: Assemble Final Model and Print Summary

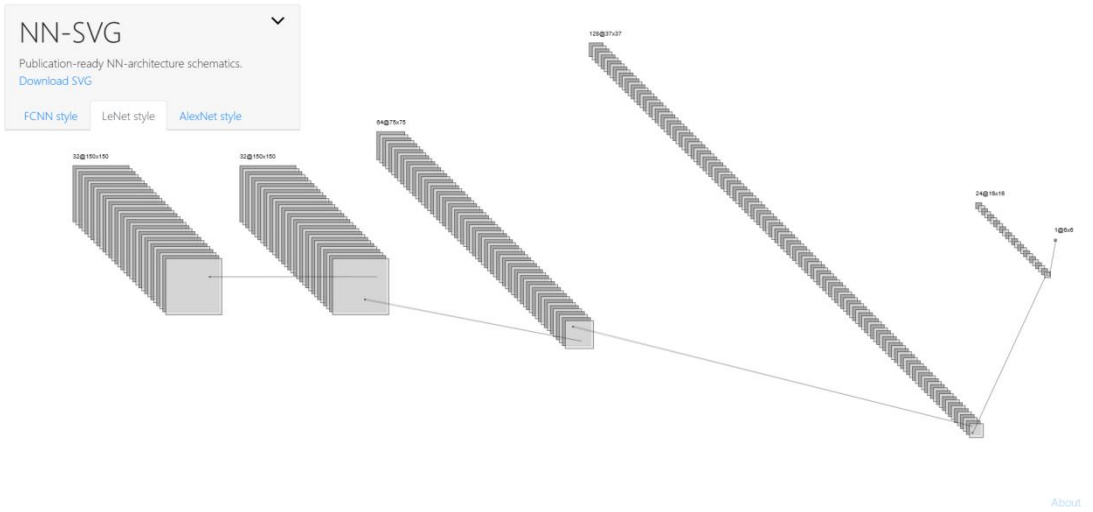


Figure 28: 3-Block CNN Architecture for Image Classification.

3.2 Training and Evaluation

```python
#---- SECTION 5.1: ADAPTIVE CNN TRAINING ----#
import tensorflow as tf
#-- 1. Compile the model. --#
model.compile(
    optimizer=tf.keras.optimizers.Adam(
        learning_rate=0.001
    ),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
# 2. Save the best model based on validation loss
checkpoint = tf.keras.callbacks.ModelCheckpoint(filepath="Base_CNN_Model.keras", monitor="val_loss", mode="min", save_best_only=True, verbose=1)
# 3. Stop automatically when validation loss stops improving
early_stopping = tf.keras.callbacks.EarlyStopping(monitor="val_loss", mode="min", patience=7, min_delta=0.001, restore_best_weights=True, verbose=1)
# 4. Reduce learning rate when learning plateaus. --#
reduce_lr = tf.keras.callbacks.ReduceLROnPlateau(monitor="val_loss", mode="min", factor=0.5, patience=3, min_lr=1e-7, verbose=1)
#-- Train without a fixed epoch limit. --#

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=1000,  # Safety limit, not a target
    callbacks=[
        checkpoint,
        reduce_lr,
        early_stopping
    ]
)

print("Training completed.")
```

Figure 29: Training Model

```
Epoch 3: val_loss improved from 0.84226 to 0.70223, saving model to Base_CNN_Model.keras

Epoch 3: finished saving model to Base_CNN_Model.keras
352/352 ━━━━━━━━━━━━━━━━━━━━ 827s 2s/step - accuracy: 0.8481 - loss: 0.4305 - val_accuracy: 0.7626 - val_loss: 0.7022 - learning_rate: 0.0010
Epoch 4/1000
352/352 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.8773 - loss: 0.3505
Epoch 4: val_loss did not improve from 0.70223
352/352 ━━━━━━━━━━━━━━━━━━━━ 821s 2s/step - accuracy: 0.8859 - loss: 0.3216 - val_accuracy: 0.7683 - val_loss: 0.7033 - learning_rate: 0.0010
Epoch 5/1000
352/352 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.9123 - loss: 0.2595
Epoch 5: val_loss did not improve from 0.70223
352/352 ━━━━━━━━━━━━━━━━━━━━ 824s 2s/step - accuracy: 0.9238 - loss: 0.2295 - val_accuracy: 0.7500 - val_loss: 0.7970 - learning_rate: 0.0010
Epoch 6/1000
352/352 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.9204 - loss: 0.2309
Epoch 6: val_loss did not improve from 0.70223

Epoch 6: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
352/352 ━━━━━━━━━━━━━━━━━━━━ 821s 2s/step - accuracy: 0.9365 - loss: 0.1800 - val_accuracy: 0.7234 - val_loss: 1.0824 - learning_rate: 0.0010
Epoch 7/1000
352/352 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.9650 - loss: 0.0984
Epoch 7: val_loss did not improve from 0.70223
352/352 ━━━━━━━━━━━━━━━━━━━━ 845s 2s/step - accuracy: 0.9751 - loss: 0.0718 - val_accuracy: 0.8017 - val_loss: 0.7277 - learning_rate: 5.0000e-04
Epoch 8/1000
352/352 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.9799 - loss: 0.0713
Epoch 8: val_loss did not improve from 0.70223
352/352 ━━━━━━━━━━━━━━━━━━━━ 805s 2s/step - accuracy: 0.9886 - loss: 0.0412 - val_accuracy: 0.8290 - val_loss: 0.7220 - learning_rate: 5.0000e-04
Epoch 9/1000
352/352 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.9796 - loss: 0.0620
Epoch 9: val_loss did not improve from 0.70223

Epoch 9: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
352/352 ━━━━━━━━━━━━━━━━━━━━ 808s 2s/step - accuracy: 0.9889 - loss: 0.0384 - val_accuracy: 0.7967 - val_loss: 0.8069 - learning_rate: 5.0000e-04
Epoch 10/1000
352/352 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.9909 - loss: 0.0463
Epoch 10: val_loss improved from 0.70223 to 0.70223, saving model to Base_CNN_Model.keras

Epoch 10: finished saving model to Base_CNN_Model.keras
352/352 ━━━━━━━━━━━━━━━━━━━━ 803s 2s/step - accuracy: 0.9954 - loss: 0.0206 - val_accuracy: 0.8362 - val_loss: 0.7022 - learning_rate: 2.5000e-04
Epoch 10: early stopping
Restoring model weights from the end of the best epoch: 3.
Training completed.
Training completed.
```

Figure 30: Training Log

5.2 Evaluation Model.

```python
#---- SECTION 5.2: VALIDATION EVALUATION ----#
best_model = tf.keras.models.load_model("Base_CNN_Model.keras")
val_loss, val_accuracy = best_model.evaluate(val_ds)
print(f"Validation Loss: {val_loss:.4f}")
print(f"Validation Accuracy: {val_accuracy * 100:.2f}%")
```

```
87/87 ━━━━━━━━━━━━━━━━━━━━ 53s 599ms/step - accuracy: 0.8362 - loss: 0.6974
Validation Loss: 0.6974
Validation Accuracy: 83.62%
```

Figure 31: Model Evaluation

```
Classification Report on Validation Set:
              precision    recall  f1-score   support

           0       0.87      0.83      0.85       415
           1       0.95      0.97      0.96       447
           2       0.81      0.81      0.81       488
           3       0.84      0.80      0.82       500
           4       0.84      0.88      0.86       455
           5       0.88      0.91      0.90       479

    accuracy                           0.87      2784
   macro avg       0.87      0.87      0.87      2784
weighted avg       0.87      0.87      0.87      2784
```

Figure 32: Classification Report on Validation Set

3.3 Testing

5.3 Testing.

```
# 1. Evaluate the model on the Test Set
test_loss, test_accuracy = best_model.evaluate(test_ds)
print(f"\nTest Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_accuracy * 100:.2f}%")

# 2. Get detailed predictions on the Test Set for report generation
y_true_test = []
y_pred_test = []

for images, labels in test_ds:
    preds = best_model.predict(images, verbose=0)
    y_true_test.extend(np.argmax(labels.numpy(), axis=1))
    y_pred_test.extend(np.argmax(preds, axis=1))

# Print full classification report
print("\nClassification Report on Test Set:")
print(classification_report(y_true_test, y_pred_test))
```

```
94/94 ━━━━━━━━━━━━━━━━━━━━ 781s 8s/step - accuracy: 0.8277 - loss: 0.8400

Test Loss: 0.8400
Test Accuracy: 82.77%

Classification Report on Test Set:
              precision    recall  f1-score   support

           0       0.82      0.76      0.79       437
           1       0.93      0.96      0.94       474
           2       0.79      0.77      0.78       553
           3       0.80      0.76      0.78       525
           4       0.81      0.87      0.84       510
           5       0.82      0.85      0.84       501

    accuracy                           0.83      3000
   macro avg       0.83      0.83      0.83      3000
weighted avg       0.83      0.83      0.83      3000
```

Figure 33: Baseline CNN Model Evaluation Output and Classification Report on the Test Dataset.

D. Conclusion

In this lab, a Baseline Convolutional Neural Network (CNN) model was successfully constructed, trained, and evaluated on the Intel Image Classification dataset. Specifically, the baseline architecture achieved a validation accuracy of 83.62% (Validation Loss: 0.6974). Furthermore, it generalized effectively to the unseen test dataset, securing an accuracy of 82.77% with a Test Loss of 0.8400. Although the model demonstrates robust image feature extraction and overall strong classification performance, the results nevertheless indicate clear potential for further optimization. Consequently, incorporating advanced regularization and data augmentation techniques in future iterations will be essential to mitigate overfitting and improve generalization on more complex visual patterns.